



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Cyclic Temperature Conversion

CMSC 131 Lab 1

Prepared by: Rene Andre Bedonia Jocsing

Published: September 04, 2025

This laboratory assignment introduces basic arithmetic operations, register manipulation, and input/output handling in the NASM assembly language. You will implement a temperature conversion program that converts between Celsius, Fahrenheit, and Kelvin temperature scales.

Learning Objectives

By the end of this laboratory exercise, students should be able to:

- Handle user input using assembly language I/O functions
- Perform arithmetic operations using CPU registers
- Implement mathematical formulas in assembly code
- Understand integer division and automatic truncation
- Follow proper assembly code structure and documentation practices

Task

Create a program that performs the following sequence:

1. Prompt the user to input a temperature in degrees Celsius.
2. Convert the Celsius value to Fahrenheit and display the result.
3. Convert the Fahrenheit value to Kelvin and display the result.
4. Convert the Kelvin value back to Celsius and display the result.

Conversion Formulas

The mathematical relationships between the temperature scales for this assignment are:

- **Celsius to Fahrenheit:** $F = (C \times 9)/5 + 32$
- **Fahrenheit to Kelvin:** $K = ((F - 32) \times 5)/9 + 273$
- **Kelvin to Celsius:** $C = K - 273$

Important Notes

- Use **unsigned integer arithmetic only** — truncate any fractional results
- Use registers appropriately — clean up of the Floating Point Unit (FPU) is not required
- Use the provided I/O functions: `print_string`, `print_int`, `read_int`
- Assume input is valid and within reasonable bounds (e.g., no negative temperatures, no absurdly large or odd inputs, etc.)
- No floating-point operations or FPU cleanup required
- A Python script is provided to validate your results

Implementation Details

Expected Output

```
1  Input a temperature in Celsius: 0
2  The temperature in Fahrenheit from Celsius is: 32
3  The temperature in Kelvin from Fahrenheit is: 273
4  The temperature in Celsius from Kelvin is: 0
5
6  Input a temperature in Celsius: 69
7  The temperature in Fahrenheit from Celsius is: 156
8  The temperature in Kelvin from Fahrenheit is: 341
9  The temperature in Celsius from Kelvin is: 68
10
11 Input a temperature in Celsius: 100
12 The temperature in Fahrenheit from Celsius is: 212
13 The temperature in Kelvin from Fahrenheit is: 373
14 The temperature in Celsius from Kelvin is: 100
```

Think: why does the result of the final conversion from Kelvin to Celsius sometimes differ from the original Celsius input?

Starter Code

A starter code template is provided along with this manual. Complete the *TODO* stubs in the code. However, following this template is not mandatory, as long as the tasks are accomplished.

Testing and Validation

Test your program with various temperature inputs to ensure accuracy:

- Room temperature (20-25°C)
- Freezing point of water (0°C)
- Boiling point of water (100°C)
- Higher temperatures (200°C, 500°C)

A Python validation script is provided to verify your results against expected outputs.

Laboratory Defense

The instructor will be available during scheduled laboratory hours in the assigned room for defense or consultation. You will be catered to at a first-come first-served basis. It is mandatory to present your code, answer inquiries, and perform live programming if the instructor deems it necessary to further check your understanding.

If a laboratory defense for an activity fails to be conducted within **one month** of its assignment, the instructor will be forced to give a failing grade. Scoring will be done during the laboratory defense (no defense, no grade policy). Extensions may be granted due to extraordinary circumstances, but ultimately remains up to the judgment of the instructor.

Academic Honesty

The usage of Large Language Models (e.g. ChatGPT, Claude, Deepseek, etc.) to generate code is considered cheating. As cheating is against the university's code of ethics, it is subject to failure in the course, harsh disciplinary action, and expulsion.

Rubric for Programming Exercises (50 pts)

Criteria (10 Points Each)	Excellent (9-10)	Good (6-8)	Fair (3-5)	Poor (0-2)
Program Correctness	Program executes correctly with no syntax or runtime errors, meets/exceeds specifications, and displays correct output	Program executes and outputs with minor errors, yet meets specifications	Program executes and outputs with major errors, yet somehow meets specifications	Program does not execute or does not meet specs
Logical Design	Program is logically well-designed with excellent structure and flow	Program has slight logic errors that do not significantly affect the results	Program has significant logic errors affecting functionality	Program logic is fundamentally incorrect
Code Mastery	Programmer demonstrates excellent mastery over the program's code	Programmer demonstrates adequate mastery over the program's code	Programmer demonstrates fair mastery over the program's code	Programmer demonstrates poor mastery over the program's code
Engineering Standards	Program is stylistically well designed from an engineering standpoint	Slight inappropriate design choices (i.e., poor variable names, improper indentation)	Severe inappropriate design choices (i.e., code repetition, redundancy)	Program is poorly written
Documentation*	Program is well-documented: comments exist for clarity, not redundancy	Missing one required comment or some redundant comments	Missing two or more required comments or many redundant comments	Most documentation missing or most documentation is redundant

***Remember: “Code tells you how, comments tell you why.”** — Jeff Atwood, co-founder of Stack Overflow and Discourse

See the [laboratory manual](#) for submission requirements.